

CXL Persistent Memory (PMEM) Emulation Report

1. Environment and Setup

The host/guest environment and QEMU/kernel build used for this track are produced by a single consolidated script, `CXL_Complete_Setup.sh`, rather than a separate manual build process.

Item	Value
Host OS	Ubuntu 24.04 LTS (native) or Ubuntu 24.04 LTS under WSL2, script auto-detects via <code>/proc/version</code>
Guest OS	Ubuntu 24.04.4 LTS (cloud image, cloud-init provisioned)
Guest Kernel	Linux v6.18, built with <code>CXL_BUS/PCI/ACPI/PMEM/MEM/PORT/REGION</code> plus <code>DAX</code> , <code>zone-device</code> , and <code>NUMA</code> support
QEMU	Custom build from the jic23 CXL fork (branch <code>cxl-2025-03-20</code>)
Firmware	OVMF, built from <code>edk2</code> source (required for ACPI CEDT table generation, SeaBIOS cannot emulate CXL)
CXL Tooling	<code>cxl-cli</code> (<code>cxl</code>), <code>ndctl</code> , <code>pciutils</code> , installed automatically if missing

Setup runs as eight ordered, idempotent phases: dependency installation, a WSL2-only kernel rebuild (skipped on native Ubuntu), the QEMU build, the guest kernel build, OVMF firmware build, backing-file and guest-image preparation, generation of quick-start launch scripts, and a final validation pass that checks every expected artifact exists. The script also applies a small set of QEMU source patches during the build so that CXL error-injection commands are not silently masked, this is discussed only briefly here since it is not the focus of this document, and is elaborated in Section 7.

The quick-start persistent-memory launch script produced by this setup (`start-cxl-pmem.sh`) uses a single Type 3 device with a unix-socket QMP endpoint. The fuller dual-switch, four-device topology documented in this report (Sections 2–5) is built on the same QEMU binary and guest kernel, scaled up to two independent switches and pools.

2. Objective

- Emulate CXL Type 3 persistent-memory devices across a dual-switch topology, modelling two independent memory pools.
- Provision each pool as an independent CXL region and namespace, backed by real LSA (Label Storage Area) metadata.
- Format and mount each namespace as an ext4 filesystem, preferring DAX where supported.
- Validate data integrity with checksum-verified read/write I/O and confirm the two pools are functionally independent.
- Independently verify the full CXL hardware/software stack (topology, kernel modules, sysfs, NUMA, dmesg) without mutating state, and capture a self-contained proof report.

3. Topology

The persistent-memory track uses two CXL switches, each exposing two Type 3 endpoints, giving four independently addressable 1 GiB pmem devices grouped into two pools:

```

pxb-cxl (bus_nr=12)
├── cxl-rp (port 0, chassis 0, slot 0) ← Switch A
│   ├── cxl-upstream (us0)
│   │   ├── cxl-downstream (ds0a) → cxl-type3 mem0 (sn=0x10)
│   │   └── cxl-downstream (ds1a) → cxl-type3 mem1 (sn=0x11)
│   └── cxl-rp (port 1, chassis 0, slot 1) ← Switch B
│       ├── cxl-upstream (us1)
│       │   ├── cxl-downstream (ds0b) → cxl-type3 mem2 (sn=0x20)
│       │   └── cxl-downstream (ds1b) → cxl-type3 mem3 (sn=0x21)

```

Component	QEMU Device	Description
CXL Host Bridge	pxb-cxl	bus_nr=12, gateway between PCIe and CXL domains
CXL Root Ports	cxl-rp × 2	One per switch (Switch A, Switch B)
CXL Upstream Ports	cxl-upstream × 2	us0 (Switch A), us1 (Switch B)
CXL Downstream Ports	cxl-downstream × 4	Two per switch, one Type 3 device each
CXL Type 3 Devices	cxl-type3 × 4	mem0–mem3, persistent-memdev, 1 GiB each
Memory Backends	cxl-mem0..3.raw	4 × 1 GiB memory-backend-file
Label Storage Areas	cxl-lsa0..3.raw	4 × 256 MiB LSA backing files
Fixed Memory Window	cxl-fmw.0	16 GiB, 4k interleave granularity

4. Emulation Script (emulation.sh)

3.1 Pre-flight Checks

Before launch, the script verifies the QEMU binary, guest kernel, OVMF firmware pair, and guest disk image all exist and are executable/readable, failing fast with a clear error if any are missing.

3.2 Backing File Preparation

Four 1 GiB memory backends and four 256 MiB LSA files are created (idempotently, existing files are not overwritten) and permissioned 660:

```

dd if=/dev/zero of=cxl-mem0.raw bs=1M count=1024
dd if=/dev/zero of=cxl-mem1.raw bs=1M count=1024
dd if=/dev/zero of=cxl-mem2.raw bs=1M count=1024
dd if=/dev/zero of=cxl-mem3.raw bs=1M count=1024

dd if=/dev/zero of=cxl-lsa0.raw bs=1M count=256
dd if=/dev/zero of=cxl-lsa1.raw bs=1M count=256
dd if=/dev/zero of=cxl-lsa2.raw bs=1M count=256
dd if=/dev/zero of=cxl-lsa3.raw bs=1M count=256

chmod 660 cxl-mem*.raw cxl-lsa*.raw

```

3.3 QEMU Launch Command

QEMU is launched with KVM acceleration, 8 GiB base RAM (maxmem=32G), and clflushopt/clwb explicitly exposed to the guest CPU but masked out at the kernel command line via clearcpuid, this

combination is required because the CXL pmem path issues cache-flush instructions, but exposing them unmasked to a migratable=off KVM guest can fault:

```
-machine q35,cxl=on,accel=kvm
-cpu host,migratable=off,+clflushopt,+clwb
-smp 4
-m 8G,maxmem=32G,slots=8

-kernel "${KERNEL}"
-append "root=/dev/vda1 rootwait rootdelay=5 \
 console=ttyS0,115200 earlyprintk=ttyS0 rw \
 clearcpuid=clflushopt,clwb"

-drive file="${GUEST_DISK}",format=qcow2,if=none,id=hd0
-device virtio-blk-pci,drive=hd0,bus=pcie.0
-netdev user,id=net0,hostfwd=tcp::2222-:22
-device virtio-net-pci,netdev=net0,bus=pcie.0
-virtfs local,path="${REPORTS_DIR}",mount_tag=hostshare,security_model=mapped-xattr,id=share
-nographic -d guest_errors

-object memory-backend-file,id=cxl-mem0,share=on,mem-path="${IMAGES}/cxl-mem0.raw",size=1G
-object memory-backend-file,id=cxl-lsa0,share=on,mem-path="${IMAGES}/cxl-lsa0.raw",size=256M
-object memory-backend-file,id=cxl-mem1,share=on,mem-path="${IMAGES}/cxl-mem1.raw",size=1G
-object memory-backend-file,id=cxl-lsa1,share=on,mem-path="${IMAGES}/cxl-lsa1.raw",size=256M
-object memory-backend-file,id=cxl-mem2,share=on,mem-path="${IMAGES}/cxl-mem2.raw",size=1G
-object memory-backend-file,id=cxl-lsa2,share=on,mem-path="${IMAGES}/cxl-lsa2.raw",size=256M
-object memory-backend-file,id=cxl-mem3,share=on,mem-path="${IMAGES}/cxl-mem3.raw",size=1G
-object memory-backend-file,id=cxl-lsa3,share=on,mem-path="${IMAGES}/cxl-lsa3.raw",size=256M
```

```
-device pxb-cxl,bus_nr=12,bus=pcie.0,id=cxl.1

-device cxl-rp,port=0,bus=cxl.1,id=rp0,chassis=0,slot=0
-device cxl-upstream,bus=rp0,id=us0
-device cxl-downstream,port=0,bus=us0,id=ds0a,chassis=1,slot=1
-device cxl-downstream,port=1,bus=us0,id=ds1a,chassis=1,slot=2
-device cxl-type3,bus=ds0a,persistent-memdev=cxl-mem0,lsa=cxl-lsa0,id=cxl-pmem0,sn=0x10
-device cxl-type3,bus=ds1a,persistent-memdev=cxl-mem1,lsa=cxl-lsa1,id=cxl-pmem1,sn=0x11

-device cxl-rp,port=1,bus=cxl.1,id=rp1,chassis=0,slot=1
-device cxl-upstream,bus=rp1,id=us1
-device cxl-downstream,port=0,bus=us1,id=ds0b,chassis=2,slot=1
-device cxl-downstream,port=1,bus=us1,id=ds1b,chassis=2,slot=2
-device cxl-type3,bus=ds0b,persistent-memdev=cxl-mem2,lsa=cxl-lsa2,id=cxl-pmem2,sn=0x20
-device cxl-type3,bus=ds1b,persistent-memdev=cxl-mem3,lsa=cxl-lsa3,id=cxl-pmem3,sn=0x21

-M cxl-fmw.0.targets.0=cxl.1,cxl-fmw.0.size=16G,cxl-fmw.0.interleave-granularity=4k

-qmp unix:/tmp/qmp-sock,server=on,wait=off
```

A QMP unix-socket control channel is opened on every launch (stale sockets are removed first) so the host can issue monitor commands against the running instance if needed.

5. Operations Script (operations.sh)

Once the guest is up, an eight-step in-guest workflow provisions both pools end to end and proves data integrity and pool independence.

Step 0a, Tool Installation

Checks for cxl (cxl-cli), ndctl, and lspci (pciutils); installs whichever are missing via apt-get.

Step 0b, CPU Flag Guard

Before any pmem operation, the script confirms clflushopt is not present in /proc/cpuinfo, i.e., that the host's clearcpuid masking (Section 3.3) took effect. If clflushopt is still exposed, the script aborts immediately, since proceeding would risk a KVM emulation fault during cache-flush-heavy pmem writes.

```
if grep -qw "clflushopt" /proc/cpuinfo; then
    die "FATAL: 'clflushopt' is still exposed to the guest."
fi
```

Step 1, Topology Verification

Confirms the full bus/memdev/port/decoder hierarchy (cxl list -BMPD), the four expected memdevs (cxl list -M), the root decoder spanning the full fixed memory window (cxl list -D -d root), and the PCIe device tree (lspci -tv).

Step 2, Create Two Independent Regions

All CXL memdevs are enabled, then two 2-way interleaved 1 GiB pmem regions are created, Region A from mem0+mem1 (Switch A) and Region B from mem2+mem3 (Switch B), both against the root decoder:

```
sudo cxl enable-memdev all

sudo cxl create-region -d decoder0.0 -m mem0 -m mem1 -w 2 -g 4096 -s 1G
sudo cxl create-region -d decoder0.0 -m mem2 -m mem3 -w 2 -g 4096 -s 1G
```

Step 3, Create Namespaces

One namespace is created per region (ndctl create-namespace --region=regionN), producing /dev/pmem0 (region0) and /dev/pmem1 (region1).

Step 4, Format as ext4

```
sudo mkfs.ext4 -F -L "cxl-pool-a" /dev/pmem0
sudo mkfs.ext4 -F -L "cxl-pool-b" /dev/pmem1
```

Step 5, Mount Both Pools

Both filesystems are mounted at /mnt/cxl-pool-a and /mnt/cxl-pool-b, preferring DAX mode (-o dax) for direct-access I/O bypassing the page cache. If a DAX mount fails, the script falls back to a normal mount automatically rather than aborting, DAX requires an fsdax-mode namespace and the clflushopt masking already checked in Step 0b.

Step 6, Write and Verify Checksums

Each pool receives a 50 MiB `/dev/urandom` file, a plain-text marker identifying which pool/switch it belongs to, and a sha256sum manifest. Each file is then re-verified in place with `sha256sum -c`, and the script dies immediately on any checksum mismatch, this is the primary data-integrity gate of the whole workflow.

Step 7, Confirm Pool Independence

The two pools' checksums are compared directly; because both were filled with independent random data, the checksums must differ. A match would indicate the pools are aliased onto the same physical backing store rather than being truly independent, and is flagged as a warning for investigation.

Step 8, Cleanup

Both filesystems are unmounted. The underlying `.raw` backing files on the host retain their contents, the whole point of the persistent-memory track, so a fresh guest boot and re-mount would still see the previously written data.

6. Guest Verification Script (`guest_verification.sh`)

This script is deliberately read-only: it never creates regions, namespaces, or filesystems. Its purpose is to independently confirm the state of a running guest and produce an evidence bundle, without depending on or interfering with `operations.sh`.

- Mounts the VirtFS host-share (`hostshare` → `/mnt/hostshare`) so captured reports land on the host filesystem.
- Records the guest kernel version (`uname -r`).
- Enumerates the full CXL topology: `cxl list -BMPD, -M, -B, -P, -D, -R`.
- Captures the PCIe device tree (`lspci -tv`) and CXL-specific PCI capabilities (`lspci -vvv | grep -i cxl`).
- Confirms which CXL kernel modules are loaded (`cxl_core`, `cxl_pci`, `cxl_mem`, `cxl_pmem`, `cxl_port`) and dumps their module parameters.
- Lists the sysfs CXL device tree (`/sys/bus/cxl/devices/`) and greps the CXL reservation out of `/proc/iomem`.
- Captures NUMA topology (`numactl --hardware`).
- Filters `dmesg` for CXL-, `ACPI0016`-, `_OSC`-, `PCI-bus`-, `pmem`-, `nvdimm`-, `cache`-, and MCE-related entries (first 80 matches).
- Records CXL-relevant `/proc/meminfo` fields (`MemTotal`, `MemFree`, `MemAvailable`, `HugePages`).
- Writes a self-contained `/root/cxl_capture.sh` proof script into the guest, then immediately executes it, producing a timestamped `cxl_proof_*.txt` report bundling kernel version, full topology, PCIe tree, sysfs devices, NUMA layout, and filtered `dmesg`, independent, reproducible evidence that does not require `operations.sh` to have run first.

7. Results Summary

Check	Result
CPU flag guard	clflushopt confirmed masked from guest before any pmem write

Check	Result
Regions	region0 (mem0+mem1) and region1 (mem2+mem3), 1 GiB each, 2-way interleave, decode_state=commit
Namespaces	namespace0.0 → /dev/pmem0, namespace1.0 → /dev/pmem1
Filesystems	ext4 on both pools, labels cxl-pool-a / cxl-pool-b
Mount mode	DAX (-o dax) preferred, automatic fallback to normal mode on failure
Data integrity	sha256sum -c PASSED on both 50 MiB test files
Pool independence	Checksums differ between Pool A and Pool B, confirmed independent
Persistence	Backing .raw files on host retain data after guest unmount
Independent verification	Full topology, kernel modules, sysfs tree, NUMA, and dmesg captured read-only into a proof report

8. CCI / QMP Error-Injection Testing Dependency

A meaningful RAS test for CXL is an inject-then-detect loop: an error or poison condition is injected at the CXL Component Command Interface (CCI), the device mailbox, typically driven from the host side via QMP, and its effect must then be observable and reportable from inside the guest (via cxl/ndctl tooling, sysfs, or dmesg). Whether this loop behaves the same for persistent (pmem) and volatile (vmem) devices depends on which stage of the workflow is being tested: device emulation plus region creation, or what happens to that region afterwards.

8.1 Stage 1, Device Emulation Through Region Creation: PMEM and VMEM Are Symmetric

Up to and including cxl create-region, the CCI/QMP error-injection and detection path is effectively identical for both memory types, because it operates below the layer where pmem and vmem diverge:

- The CXL poison list and error-record state live in the device's own CCI mailbox, implemented by QEMU's cxl-type3 device model. This mailbox exists identically whether the device has persistent-memdev= or volatile-memdev= attached, it is not a function of the LSA or of any namespace/filesystem layer.
- QMP injection calls target a specific device id (e.g., cxl-pmem0..3 or cxl-dev0..3 in the scripts documented here) and act at a Device Physical Address (DPA) offset inside that device's mailbox state, the same call shape and same DPA addressing scheme apply to both.
- Region creation itself (cxl create-region, whether default pmem type or -t ram for vmem) still routes through the same decoder → device → DPA chain on both tracks. Creating a region does not change how the poison list is stored or queried, it only defines how the device's capacity is exposed as an addressable range.
- Detection at this stage, cxl list -l to read back injected poison, or the equivalent mailbox Get Poison List command, works the same way for a freshly created pmem region and a freshly created vmem region, because both are still fully within CXL/nvdimmem-aware territory and neither has yet been handed off to a different kernel subsystem.
- Practical conclusion: if a RAS test plan stops at "emulate Type 3 device, create region, inject error, detect it," pmem and vmem give equivalent, symmetric results in this environment. There is no meaningful CCI/QMP behavioral difference to design around at this stage.

8.2 Stage 2, What Happens After Region Creation: PMEM and VMEM Diverge

The asymmetry appears only once each track proceeds to its normal next step, because pmem and vmem are handed to different parts of the Linux kernel from that point on.

- **PMEM path** (operations.sh, persistent track): the region is provisioned as an ndctl namespace (Step 3), formatted ext4, and mounted, normally DAX. This entire path stays inside the CXL/nvdimmm subsystem: namespace, badblocks, and poison-list state are all tracked through libnvdimm and cxl_pmem. cxl list -l and ndctl continue to see the device's poison/error state throughout, and can be cross-referenced against the sha256sum-verified files written in Step 6.
- **VMEM path** (operations.sh, volatile track): the region is instead onlined directly into the kernel's general-purpose System RAM pool via sysfs (echo online > /sys/devices/system/memory/memory*/state, Step 3 of the volatile workflow). At that point the capacity stops being represented as a distinct CXL/nvdimmm object and becomes ordinary page-allocator RAM, indistinguishable from any other System RAM to the rest of the kernel.
- **Consequence for detection:** once vmem capacity is onlined, CXL-specific detection tooling (cxl list -l, ndctl badblocks) no longer has visibility into it, because that memory has left CXL/nvdimmm-tracked territory. A poison record injected before onlining may still be queryable at the device/mailbox level, but there is no longer a kernel-side consumer surfacing it against the memory now in general use.
- **Shared limitation regardless of stage:** real hardware would also expect an uncorrectable error to surface as a Machine Check Exception via EDAC/MCE. guest_verification.sh's dmesg capture shows cxl_pci ...: CXL MCE unsupported on every device on both tracks, confirming QEMU does not yet deliver injected errors as MCEs to the guest, so MCE-driven detection is unavailable either way, independent of the pmem/vmem split described above.

8.3 Summary

Workflow Stage	PMEM	VMEM	Same or Different?
Device emulation (cxl-type3, CCI mailbox)	Mailbox present, DPA-addressed	Mailbox present, DPA-addressed	Same
Region creation (cxl create-region)	Routed through decoder → device → DPA	Routed through decoder → device → DPA	Same
QMP error/poison injection	Targets device id + DPA offset	Targets device id + DPA offset	Same
Detection immediately after region creation	cxl list -l visible	cxl list -l visible	Same
State after normal next step	Namespace + ext4/DAX, stays in CXL/nvdimmm tracking	Onlined into generic System RAM, leaves CXL/nvdimmm tracking	Different
Detection after normal next step	cxl list -l / ndctl still see poison/error state	No CXL-aware consumer once onlined	Different
MCE-based detection	Unsupported by QEMU (dmesg confirms)	Unsupported by QEMU (dmesg confirms)	Same (both unavailable)

Practical implication: CCI/QMP error-injection coverage of the device and region-creation layer is equally valid on either track, so both can be used interchangeably to exercise and validate the injection path itself. Once a test needs the injected condition to be detectable after the region is put to its normal use, the pmem track should be used, since only it keeps the memory inside CXL/nvdimmm-aware tooling; the

vmem track's detection window effectively closes as soon as the region is onlined into general System RAM.